

EXPERIMENT 07:

6. Construct a C program to implement pre-emptive priority scheduling algorithm.

```

1 #include <stdio.h>
2
3 int main()
4 {
5     int n,i,j,temp;
6     int bt[20],wt[20],tat[20];
7
8     printf("Enter Number of Processes: ");
9     scanf("%d",&n);
10
11     for(i=0;i<n;i++)
12     {
13         printf("Burst Time of P%d: ",i+1);
14         scanf("%d",&bt[i]);
15     }
16
17     for(i=0;i<n-1;i++)
18     {
19         for(j=i+1;j<n;j++)
20         {
21             if(bt[i]>bt[j])
22             {
23                 temp=bt[i];
24                 bt[i]=bt[j];
25                 bt[j]=temp;
26             }
27         }
28     }
29
30     wt[0]=0;
31
32     for(i=1;i<n;i++)
33     {
34         wt[i]=wt[i-1]+bt[i-1];
35     }
36
37     for(i=0;i<n;i++)
38     {
39         printf("Process: %d\tWT: %d\tTAT: %d\n",i+1,wt[i],wt[i]+bt[i]);
40     }
41 }

```

Run

3
P1 3
P2 4
P3 4

Output

Status: Successfully executed

Time:	Memory:
0.0000 secs	1.784 Mb

Sample Input

```

3
P1 3
P2 4
P3 4

```

Your Output

```

Enter Number of Processes: Burst Time of P1: Burst Time of P2: Burst
Process BT WT TAT

```

PSEUDO CODE :

Start

Input processes

Input Arrival Time

Input Burst Time

Input Priority

Repeat until all processes finish

Select highest priority process

Execute for one unit

If higher priority process arrives

Switch process

Update Remaining Time

Calculate Waiting Time

Calculate Turnaround Time

Display results

Stop

Process AT BT Priority WT TAT

P1 0 5 2 5 10

P2 1 3 1 0 3

P3 2 4 3 6 10

Average Waiting Time = 3.67

Average Turnaround Time = 7.67

CODE:

```
#include <stdio.h>

int main()
{
    int n,i,j,temp;
    int bt[20],wt[20],tat[20];

    printf("Enter Number of Processes: ");
    scanf("%d",&n);

    for(i=0;i<n;i++)
    {
        printf("Burst Time of P%d: ",i+1);
        scanf("%d",&bt[i]);
    }

    for(i=0;i<n-1;i++)
    {
        for(j=i+1;j<n;j++)
        {
            if(bt[i]>bt[j])
            {
                temp=bt[i];
                bt[i]=bt[j];
                bt[j]=temp;
            }
        }
    }

    wt[0]=0;

    for(i=1;i<n;i++)
```

```
wt[i]=wt[i-1]+bt[i-1];
```

```
printf("\nProcess\tBT\tWT\tTAT\n");
```

```
for(i=0;i<n;i++)
```

```
{
```

```
    tat[i]=wt[i]+bt[i];
```

```
    printf("P%d\t%d\t%d\t%d\n",i+1,bt[i],wt[i],tat[i]);
```

```
}
```

```
return 0;
```

```
}
```

